

Write a C program to simulate Real-Time CPU scheduling algorithms:

- Rate-Monotonic
- Earliest-Deadline First
- ~~Proportional Scheduling~~

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Process
```

```
{
    int id;
```

```
    int burst;
```

```
    int deadline;
```

```
    int period;
```

```
    int ct, wt, tat;
```

```
};
```

```
void EDF (struct Process p[], int n) {
```

```
    printf("In Earliest Deadline First (EDF) scheduling\n");
```

```
    printf("ID\tBT\tDeadline\tCT\tWT\tTAT\n");
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (p[i].deadline > p[j].deadline) {
```

```
                struct Process temp = p[i];
```

```
                p[i] = p[j];
```

```
                p[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```

int time = 0;
for (int i = 0; i < n; i++) {
    time += p[i].burst;
    p[i].ct = time;
    p[i].tat = p[i].ct;
    p[i].wt = p[i].tat - p[i].burst;
}

```

```

printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id,
       p[i].burst, p[i].deadline, p[i].ct, p[i].wt,
       p[i].tat);
}
}

```

```

void RMS (struct Process p[], int n) {
    printf("In Rate Monotonic Scheduling (RMS) \n");
    printf("ID\tB\tP\tCT\tWT\tTAT\n");
}

```

```

for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (p[i].period > p[j].period) {
            struct Process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

```

```

int time = 0;

for (int i = 0; i < n; i++) {
    time += p[i].burst;
    p[i].ct = time;
}

```



```

p[i].tat = p[i].ct;
p[i].wt = p[i].tat - p[i].burst;

```

```

printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].burst,
      p[i].period, p[i].ct, p[i].wt, p[i].tat);

```

```

}

```

```

}

```

```

int main() {

```

```

    int n;

```

```

    printf("Enter number of processes: ");

```

```

    scanf("%d", &n);

```

```

    struct Process p[n], p1[n], p2[n];

```

```

    printf("%d", &n); // Enter process details: n

```

```

    for (int i = 0; i < n; i++) {

```

```

        p[i].id = i;

```

```

        printf("In Process: %d\n", i);

```

```

        printf("Burst Time: ");

```

```

        scanf("%d", &p[i].burst);

```

```

        printf("Deadline (for EDF): ");

```

```

        scanf("%d", &p[i].deadline);

```

```

        printf("Period (for RMS): ");

```

```

        scanf("%d", &p[i].period);

```

```

    }

```

```

    memcpy(p1, p, sizeof(p));

```

```

    memcpy(p2, p, sizeof(p));

```

```

    EDF(p1, n);

```

```

    RMS(p2, n);

```

```

    return 0;

```

```

}

```

output :

Enter number of processes : 3

Enter process details:

Process 0:

Burst Time : 5

Deadline (for EDF) : 2

Period (for RMS) : 5

Process 1:

Burst Time : 4

Deadline (for EDF) : 7

Period (for RMS) : 3

Process 2:

Burst Time : 8

Deadline (for EDF) : 5

Period (for RMS) : 4

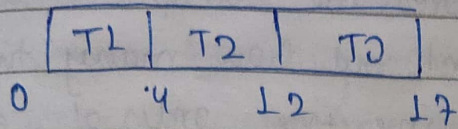
Earliest Deadline First (EDF) Scheduling

ID	BT	Deadline	CT	WT	TAT
0	5	2	5	0	5
2	8	5	13	5	13
1	4	7	17	13	17

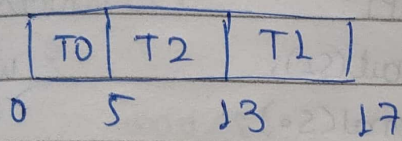
Rate Monotonic Scheduling (RMS)

ID	BT	Period	CT	WT	TAT
1	4	3	4	0	4
2	8	4	12	4	12
0	5	5	17	12	17

Gantt Chart:
(RMS)



~~Gantt~~ EDF



~~LCM for (3, 4, 5) =~~

~~60~~
31/03/26